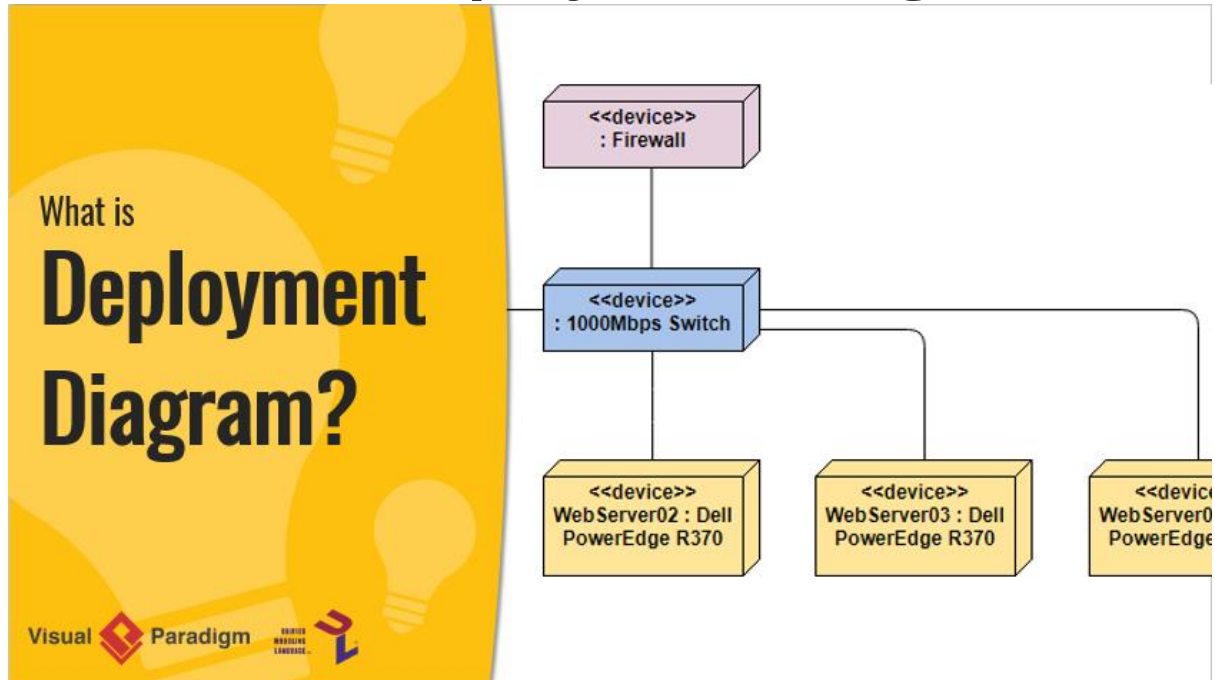
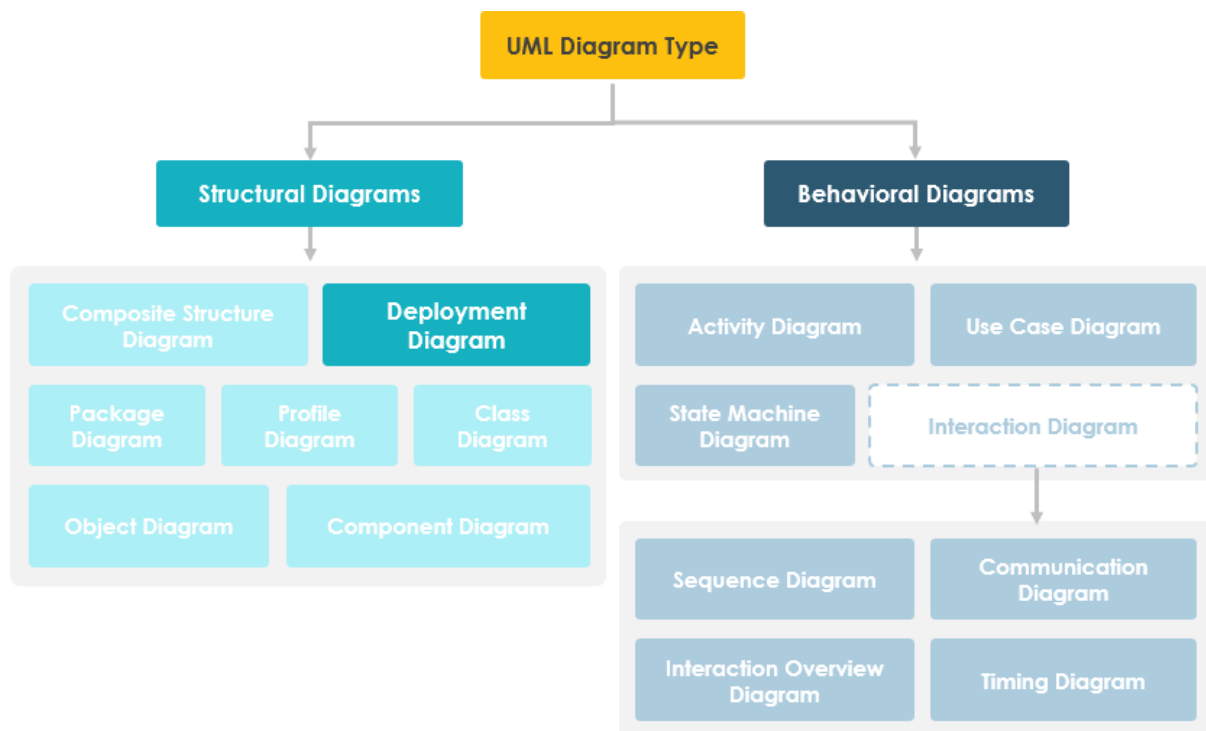


What is Deployment Diagram?



<https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-deployment-diagram/>

A **UML** deployment diagram is a diagram that shows the configuration of run time processing nodes and the components that live on them. Deployment diagrams is a kind of structure diagram used in modeling the physical aspects of an object-oriented system. They are often be used to model the static deployment view of a system (topology of the hardware).



When to Use Deployment Diagram

- What existing systems will the newly added system need to interact or integrate with?
- How robust does system need to be (e.g., redundant hardware in case of a system failure)?
- What and who will connect to or interact with system, and how will they do it
- What middleware, including the operating system and communications approaches and protocols, will system use?
- What hardware and software will users directly interact with (PCs, network computers, browsers, etc.)?
- How will you monitor the system once deployed?
- How secure does the system needs to be (needs a firewall, physically secure hardware, etc.)?

Purpose of Deployment Diagrams

- They show the structure of the run-time system
- They capture the hardware that will be used to implement the system and the links between different items of hardware.

- They model physical hardware elements and the communication paths between them
- They can be used to plan the architecture of a system.
- They are also useful for Document the deployment of software components or nodes

Deployment Diagram at a Glance

Deployment diagrams are important for visualizing, specifying, and documenting embedded, client/server, and distributed systems and also for managing executable systems through forward and reverse engineering.

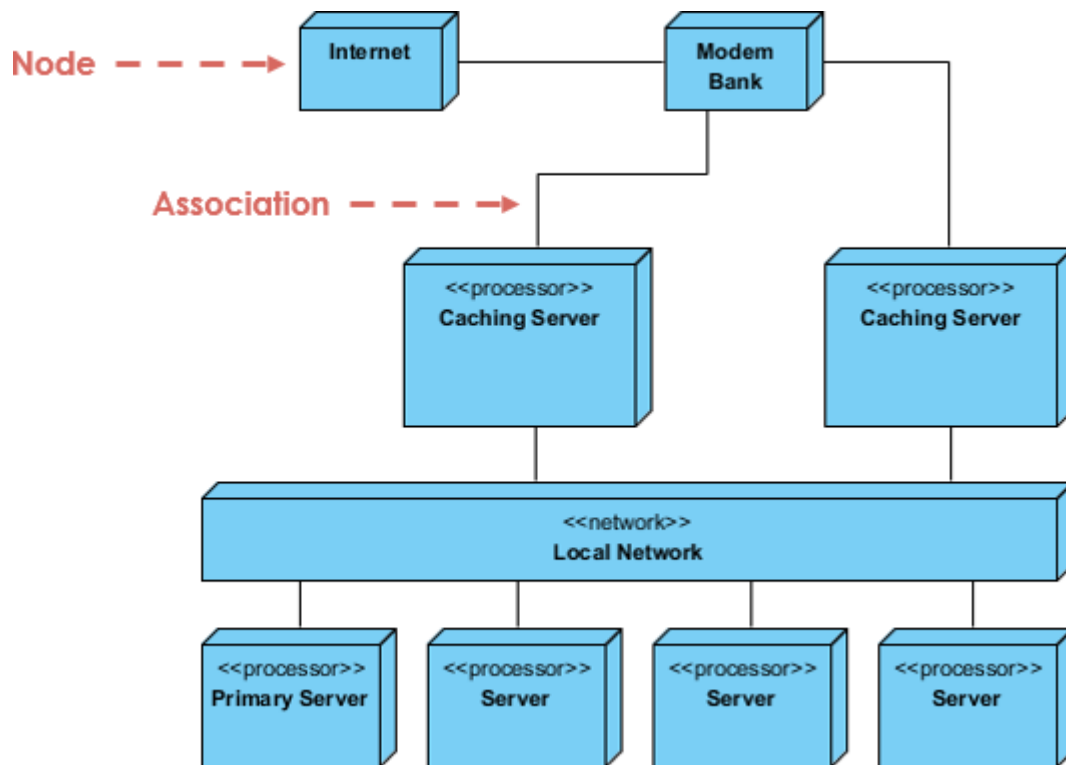
A deployment diagram is just a special kind of class diagram, which focuses on a system's nodes. Graphically, a deployment diagram is a collection of vertices and arcs. Deployment diagrams commonly contain:

Nodes

- 3-D box represents a node, either software or hardware
- HW node can be signified with <<stereotype>>
- Connections between nodes are represented with a line, with optional <<stereotype>>
- Nodes can reside within a node

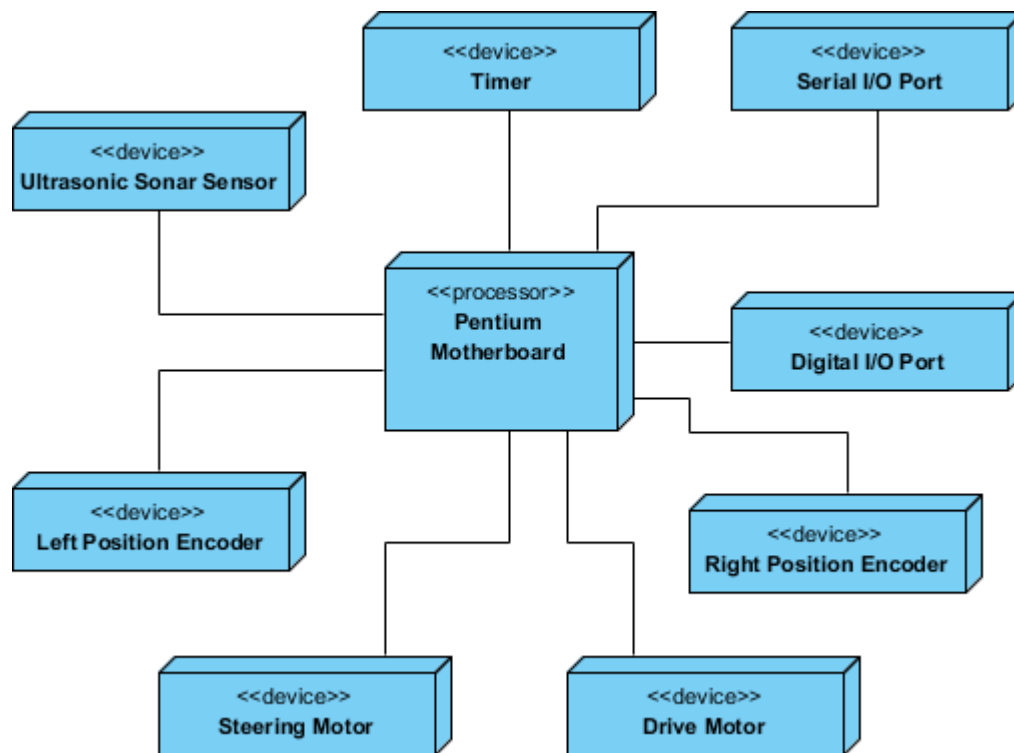
Other Notations

- Dependency
- Association relationships.
- May also contain notes and constraints.



Steps for Modeling an Embedded System

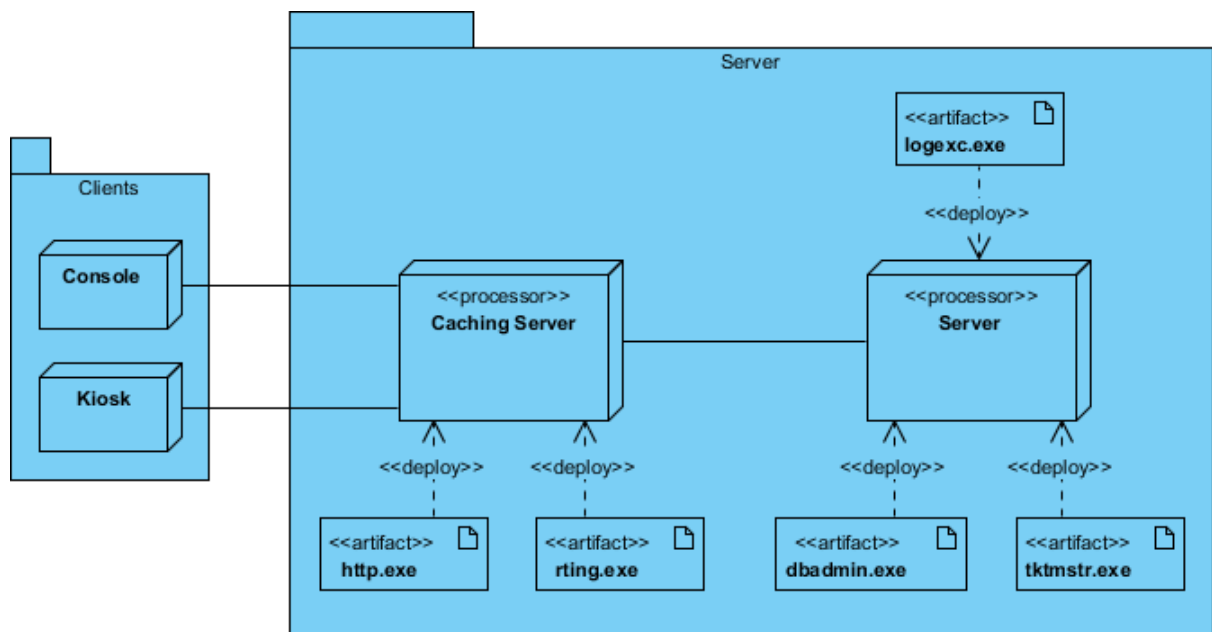
1. Identify the devices and nodes that are unique to your system.
2. Provide visual cues, especially for unusual devices, by using the UML's extensibility mechanisms to define system-specific **stereotypes** with appropriate icons. At the very least, you'll want to distinguish processors (which contain software components) and devices (which, at that level of abstraction, don't directly contain software).
3. Model the relationships among these processors and devices in a deployment diagram. Similarly, specify the relationship between the components in your system's implementation view and the nodes in your system's deployment view.
4. As necessary, expand on any intelligent devices by modeling their structure with a more detailed deployment diagram.



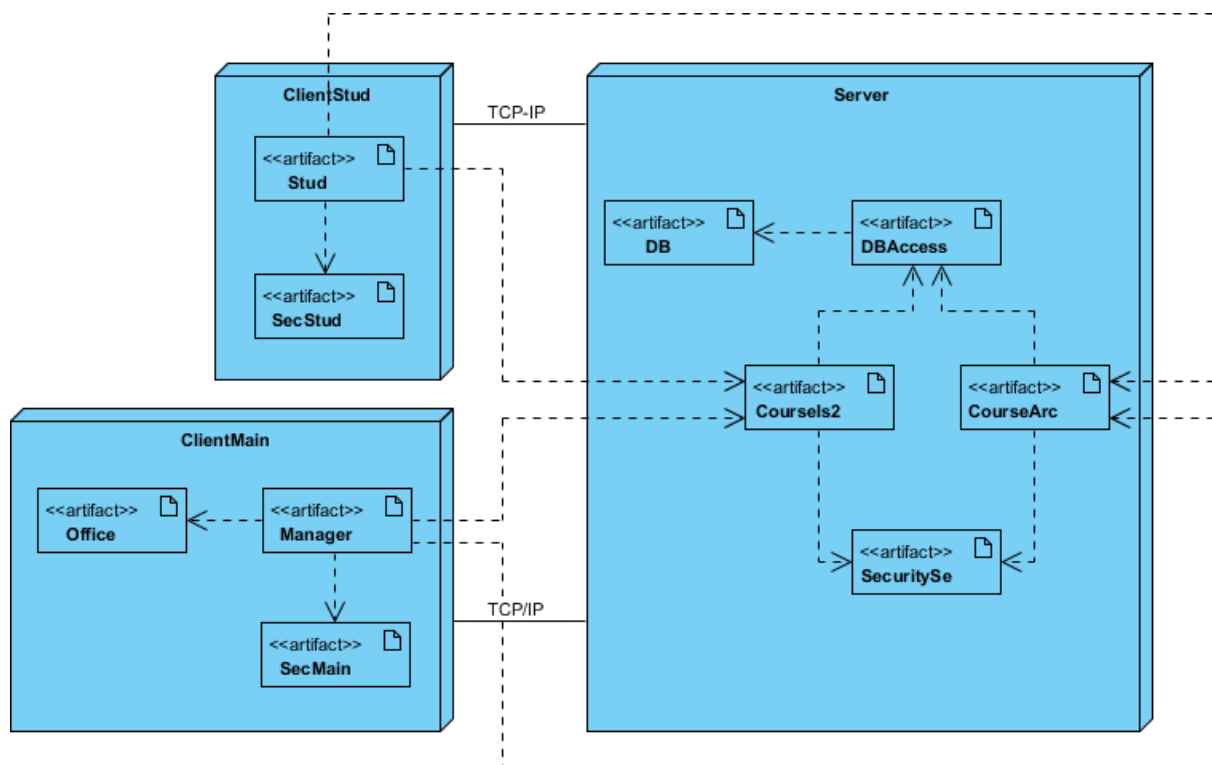
Steps for Modeling a Client/Server System

1. Identify the nodes that represent your system's client and server processors.
2. Highlight those devices that are germane to the behavior of your system. For example, you'll want to model special devices, such as credit card readers, badge readers, and display devices other than monitors, because their placement in the system's hardware topology are likely to be architecturally significant.
3. Provide visual cues for these processors and devices via stereotyping.
4. Model the topology of these nodes in a deployment diagram. Similarly, specify the relationship between the components in your system's implementation view and the nodes in your system's deployment view.

The example shows the topology of a human resources system, which follows a classical client/server architecture.



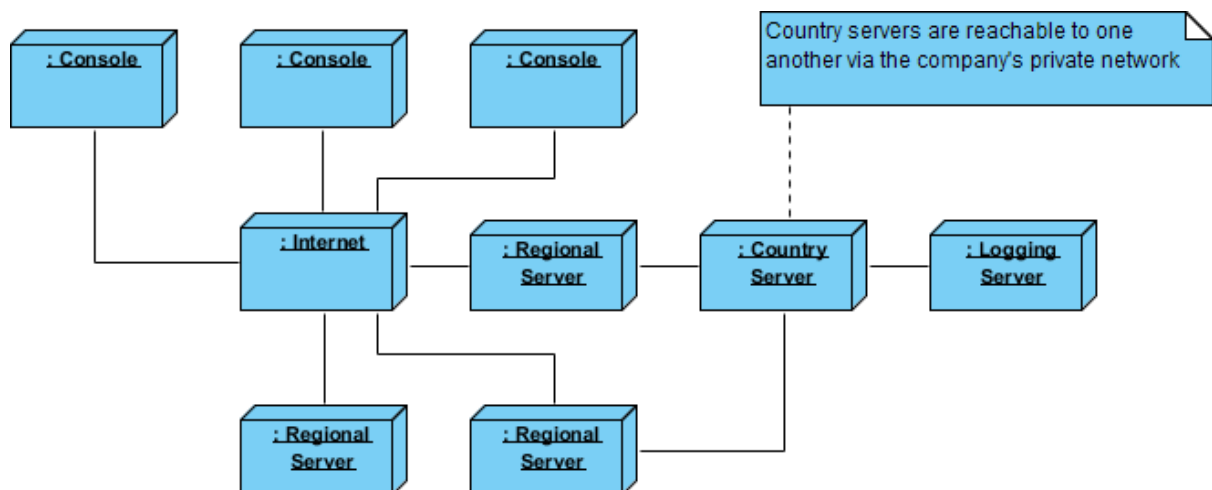
TCP/IP Client / Server Example



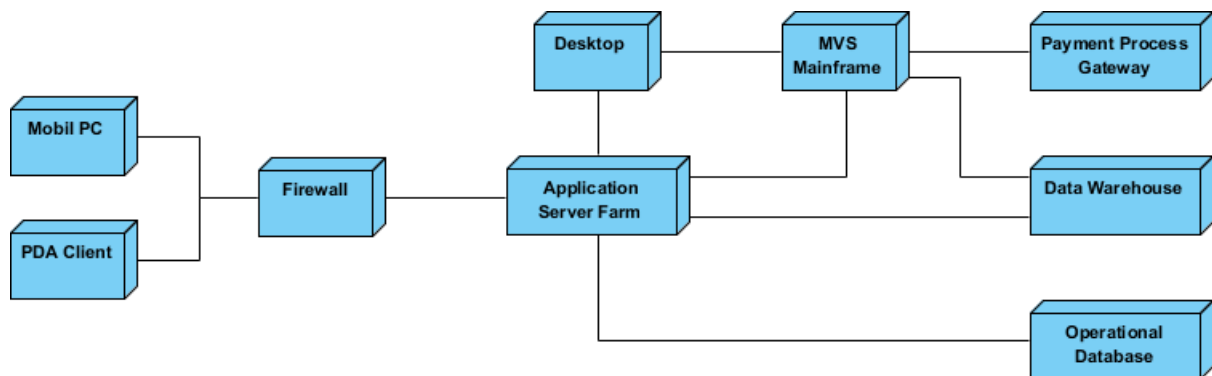
Deployment Diagram Example - Modeling a Distributed System

1. Identify the system's devices and processors as for simpler client/server systems.
2. If you need to reason about the performance of the system's network or the impact of changes to the network, be sure to model these communication devices to the level of detail sufficient to make these assessments.
3. Pay close attention to logical groupings of nodes, which you can specify by using packages.
4. Model these devices and processors using deployment diagrams. Where possible, use tools that discover the topology of your system by walking your system's network.
5. If you need to focus on the dynamics of your system, introduce use case diagrams to specify the kinds of behavior you are interested in, and expand on these use cases with interaction diagrams.
6. When modeling a fully distributed system, it's common to reify the network itself as a node. i.e. Internet, LAN, WAN as nodes

The Example shows the topology of a fully distributed system.



Deployment Diagram Example - Corporate Distributed System



Deployment Planning Checklist

When you are drafting a deployment planning for your company, you may find that you do not know where to start or what you should focus on. The following checklist may give you some ideas with planning for deployment:

- **How will your system be installed?**
 1. Who will install it? How long should it take to install?
 2. Where the installation possibly fail?
 3. How do you back out if the installation fails? How long does it take to back out?
 4. What is your installation window (during what time period can you install your system)?
 5. What backups do you need before installation?
 6. Do you need to do a data conversion?
 7. How do you know that the installation was successful?
- **If different versions of the system will be in production at the same time, how will you resolve differences?**
- **What physical sites do you need to deploy to and in what order?**
 1. How will you train your support and operations staff?
 2. Do you need to deploy a production support system so that the support staff uses their own environment to simulate problems?
- **How will you train your users?**
 1. What documentation, and in what formats and languages, do your users, and support and operation staff need?
 2. How will updates to documentation be deployed?